

Enabling Pick-and-Throw on Low-Torque Manipulators through Constraint-Aware Data Efficient Model-Based Learning

Anonymous submission

Abstract—Model-based reinforcement learning (MBRL) enables data-efficient robotic pick-and-throw by learning object dynamics from a small number of physical trials. However, existing approaches typically assume a predefined release configuration, which can be inadequate for manipulators with limited torque and velocity capabilities. This paper presents a constraint-aware extension of model-based pick-and-throw in which the release state is explicitly searched under kinematic, torque, and joint-velocity constraints. Instead of validating only the release instant, we evaluate the complete windup, throw, and post-release follow-through trajectories, thereby accounting for dynamic constraints that can otherwise produce infeasible releases. The resulting release-state search is integrated with a Gaussian-process dynamics model and particle-based policy optimization, while a shared release solver is used consistently for training and execution. Across four simulated manipulators, the proposed approach achieves 1.4–2.7 cm mean landing error without per-arm policy tuning beyond the corresponding torque and velocity limits. On the Kinova Gen3, the analysis identifies a measurable 0.83 m safe-throw range under the full-trajectory feasibility constraints. We further show that a single policy can generalize across a continuous range of target heights and characterize a drag regime in which model-based learning provides a $3\text{--}7\times$ accuracy improvement over a no-drag analytical baseline. Hardware experiments on the Gen3 validate the non-destructive stages of the execution pipeline, including state feedback, gripper actuation, and full-speed joint-velocity streaming; physical throwing and landing evaluation remain future work.

Index Terms—Pick-and-throw, model-based RL, Gaussian process regression, manipulator dynamics, and extrinsic dexterity

I. INTRODUCTION

Pick-and-throw (PnT) provides a complementary alternative to conventional pick-and-place manipulation. By releasing an object with a controlled velocity and allowing it to travel ballistically, a robot can place objects beyond its direct end-effector workspace while potentially reducing the time required for long transport motions. This capability is attractive for applications such as sorting, material handling, and high-throughput manipulation. However, accurate throwing requires the robot to generate a sufficiently fast release while respecting its kinematic and dynamic limits. Small errors in release position, velocity, or timing can produce substantial landing errors, particularly at high release speeds.

Recent work has shown that model-based reinforcement learning (MBRL) can make robotic throwing substantially more data efficient. MC-PILOT [1], for example, combines sparse Gaussian-process (GP) dynamics learning with particle-based policy optimization and achieves accurate throwing from only a small number of physical trials. A key design choice in that framework is to use a predefined release

configuration: the learned policy determines the throwing direction and release speed, while the joint trajectory leading to the release state is fixed analytically. This decomposition reduces the dimensionality of policy optimization and is effective on the Franka Emika Panda considered in the original study.

A fixed release configuration, however, implicitly assumes that the selected manipulator has sufficient actuator capability to realize the desired release motion. This assumption becomes restrictive for lower-torque manipulators. In our experiments with the 7-DOF Kinova Gen3, whose wrist actuators provide substantially less torque than those of the Panda, directly adopting the fixed-release strategy does not produce a useful throw. Under the predefined release posture, increasing the release elevation reduces the achievable range, causing distance-based optimization to favor a near-horizontal motion rather than a genuine throw. This observation motivates a different formulation: rather than treating the release configuration as a fixed design choice, the release state itself should be selected according to the capabilities of the robot.

We therefore formulate release-state selection as a constraint-aware search over robot posture and release direction. For each candidate release configuration, the maximum achievable release speed is obtained under joint-velocity constraints, and candidates are subsequently evaluated using a feasibility cascade that considers the complete motion. In particular, we check the windup motion, the acceleration phase leading to release, and the post-release follow-through. This whole-trajectory evaluation is important because a release state can satisfy torque constraints at the release instant while violating torque or velocity limits at intermediate points of the motion. The resulting release-state search is performed once for a given robot and workspace configuration and is then combined with the MBRL policy, which learns the target-dependent release speed from the observed throwing dynamics.

Beyond enabling the release-state formulation, our analysis reveals several properties of model-based pick-and-throw that are not captured by evaluating a single optimized policy. First, we evaluate the reliability of the baseline optimization across multiple random seeds rather than relying on a single converged run. Second, we investigate whether the learned policy can represent a continuous range of target heights instead of requiring separate policy optimization for each height. Third, we compare the learned dynamics model with a closed-form no-drag ballistic model over a range of drag conditions to identify when model-based learning provides

a meaningful advantage. These experiments separate the benefit of learning the throwing dynamics from the benefit of simply using an analytical model when the underlying assumptions are sufficiently accurate.

The resulting framework is evaluated in **rigid-body simulation** on four manipulators: KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3. **Across these platforms, the constraint-aware release-state search achieves 1.4–2.7 cm mean landing error without per-arm policy tuning beyond the corresponding torque and velocity limits. On the Gen3, whole-trajectory feasibility reduces the maximum recoverable throwing range relative to a release-instant-only analysis, revealing a measurable 0.83 m safe-throw ceiling under the considered configuration and constraints. We further demonstrate continuous target-height generalization, with a single policy achieving 2.0 cm mean landing error on previously unseen target heights. Finally, experiments over different drag regimes show that the analytical no-drag model is competitive when drag is negligible, whereas model-based learning provides a 3–7 times reduction in landing error when drag becomes significant.**

Our contributions are as follows:

- 1) Constraint-aware release-state search: We **replace** a fixed release configuration with a robot-specific search over release posture and direction, subject to joint-velocity and dynamic constraints.
- 2) Whole-trajectory feasibility analysis: We introduce a feasibility cascade that evaluates windup, throw, and post-release follow-through motions rather than checking only the release instant, and quantify its effect on the feasible throwing range.
- 3) Generalization and model-analysis study: We demonstrate **continuous** target-height generalization and characterize the operating regime in which model-based learning provides an advantage over a no-drag analytical model.
- 4) Multi-manipulator evaluation and hardware validation: We evaluate the **same release-state** and learning pipeline across four simulated manipulators and validate the **non-destructive components** of the execution pipeline on a real Kinova Gen3.

The scope of the present work is primarily **simulation-based. While the complete non-destructive execution pipeline has been validated on the real Gen3, physical throwing and landing measurements are not yet included. We therefore use the hardware experiments to validate the feasibility of the execution stack rather than to claim real-world throwing accuracy.**

II. RELATED WORK

A. Analytical throwing

Analytical throwing methods derive release conditions from a ballistic or kinematic model of the object and manipulator. Such approaches can provide fast and interpretable solutions when the relevant dynamics are accurately known, but their performance depends on the validity of the underlying

model and on the available actuator capabilities. In particular, a release configuration designed for one manipulator may not remain feasible when transferred to a robot with a different torque or velocity envelope.

B. Model-free learning

TossingBot [2] learns a throwing policy from outcome data using residual physics. Its formulation demonstrates that throwing can be learned directly from robot interaction, but requires substantially more physical trials than model-based approaches. TossingBot also fixes the release direction and learns the release speed, reducing the dimensionality of the action space. This design is motivated by the sample cost of model-free learning. In contrast, MC-PILOT [1] performs policy optimization offline against a learned dynamics model, making the computational cost of a larger action space less directly tied to the number of physical trials.

C. Model-based learning

MC-PILOT [1] combines sparse Gaussian-process dynamics learning with particle-based policy optimization and achieves data-efficient throwing on a Franka Panda. Its formulation learns the target-dependent release speed while using a predefined release configuration and a hand-designed swing trajectory. The approach also demonstrates adaptation to different target heights by re-optimizing the policy using the already learned dynamics model.

Our work addresses limitations that arise when this design is transferred to a manipulator with less actuator headroom. Specifically, we make the release state itself a search variable, impose torque and velocity constraints over the complete throwing motion, and evaluate the reliability and generalization of the resulting model-based policy across multiple manipulators, target heights, and drag conditions. The resulting formulation remains lower-dimensional than a general constrained trajectory optimizer: the search determines a release state and direction, while the subsequent motion follows fixed-shape trajectory segments.

D. Constraint-aware motion generation

Manipulator trajectory optimization commonly incorporates joint-position, velocity, acceleration, and torque constraints. Such constraints are particularly important for high-speed motions, where satisfying limits at a single configuration does not guarantee feasibility along the trajectory. Our formulation adopts this principle specifically for pick-and-throw: candidate release states are filtered using a staged feasibility cascade covering the windup, release acceleration, and post-release recovery. The objective is not to solve a general-purpose trajectory optimization problem, but to provide a computationally manageable release-state selection mechanism that can be reused during model-based policy learning.

III. CONSTRAINT-AWARE MODEL-BASED PICK-AND-THROW

A. Problem Formulation and Learning Framework

Following MC-PILOT [1], we formulate pick-and-throw as a model-based policy-search problem. A sparse Gaussian-process (GP) model approximates the object's velocity dynamics from collected transitions, and particle-based policy optimization searches for a target-conditioned release speed. The main modification is that the release configuration is no longer fixed a priori. Instead, we first determine a release state that is kinematically and dynamically feasible for the selected manipulator.

The complete pipeline therefore consists of three stages. First, a robot-specific release-state search determines a feasible release posture, direction, and maximum achievable release speed. Second, the resulting release-state table is integrated with the GP-based policy optimization. Third, the learned policy is evaluated using a rigid-body simulation of the complete arm and object dynamics rather than using the training-time particle cost alone. The same release solver is used for simulation and hardware execution.

Following [1], the object dynamics are represented by three independent sparse GPs, one for each Cartesian velocity component. Each GP is trained using transitions of the form (position, velocity) $\rightarrow$ Δ velocity collected from throwing trials. An RBF policy maps the target position, and in the height-generalized formulation the target height, to a release speed.

The policy output is constrained to $[0, u_M]$ using

$$u = \frac{u_M}{2} (\tanh(x) + 1), \quad (1)$$

where x is the unconstrained policy output. During policy optimization, particles are propagated through the learned GP dynamics and the policy is updated to minimize expected landing distance. Because optimization is performed against the learned model, policy improvement between physical trials does not require additional robot interaction.

B. Release-State Optimization

The original formulation fixes the release joint configuration q_{rel} and learns only the target-dependent release speed and aiming variables. We instead search over the release state $(q, \dot{q})$. The search has two levels: an outer enumeration over candidate release postures and directions, and an inner optimization that determines the maximum release speed achievable at each candidate.

Inner release-speed optimization. For the 7-DOF Gen3, the three pitch joints provide the dominant contribution to velocity perpendicular to the selected swing plane. The four roll/twist joints are therefore held at zero velocity during release-state optimization:

$$\begin{aligned} \max_{q, s} \quad & s \\ \text{s.t.} \quad & J(q)\dot{q} = s\hat{d}, \\ & |\dot{q}_i| \leq \dot{q}_i^{\max}, \quad \forall i, \\ & \dot{q}_i = 0, \quad i \in \{0, 2, 4, 6\}. \end{aligned} \quad (2)$$

Here, s is the release speed and $\hat{d}$ is the desired unit release direction. The problem is linear in $(\dot{q}, s)$ and is solved using `scipy.optimize.linprog` with the HiGHS solver.

The static angles of the roll/twist joints remain free search variables. They are not assigned velocities because allowing them to contribute to release motion can produce a rotational motion that is poorly aligned with the desired throwing direction. A nonzero static offset also prevents the pitch axes from becoming geometrically degenerate.

Outer posture and direction search. Candidate release postures are obtained by grid-searching the relevant joint angles and static offsets. For each posture, we first check minimum end-effector height and static gravity-induced torque feasibility. Candidate release directions are then sampled within the posture-dependent swing plane, using elevation angles from 5° to 45° at 5° increments and both elevation signs.

For every surviving posture-direction pair, the inner optimization in Eq. (2) determines the maximum feasible release speed. Candidates are subsequently evaluated by the complete feasibility cascade described in Sec. III-D. Among candidates that pass all checks, the release state that maximizes the true landing distance is selected. Landing distance is computed by forward integration of the drag-augmented ballistic dynamics from the candidate's actual release position and velocity.

C. Azimuth Propagation

Once a release state has been verified for a given arm and workspace configuration, it is propagated over the required base-azimuth range by rotating the base joint. Because gravity is invariant under rotation about the vertical axis, the corresponding Jacobian and dynamic feasibility properties rotate consistently with the base angle. We therefore construct an azimuth-to-pose table rather than independently optimizing every azimuth.

For the Gen3, the verified release state is propagated over a -33° to $+33^\circ$ azimuth range at 3° increments. A turret-aiming correction accounts for the offset between the tabulated release geometry and the target bearing. The same `OptimizedReleaseSolver` is used during training and execution, ensuring that both stages use the same release-state computation.

D. Whole-Trajectory Feasibility

Release-instant feasibility alone does not guarantee that the complete throwing motion can be executed within the manipulator's limits. We therefore evaluate each candidate using four sequential checks. A candidate is rejected at the first stage that it fails.

1) **Kinematic windup bound.** The windup configuration

$$q_{\text{windup}} = q_{\text{rel}} - \dot{q}_{\text{rel}} \frac{t_{\text{throw}}}{2} \quad (3)$$

must satisfy all joint-position limits.

2) **Windup-path torque.** We sample the neutral-to-windup motion and evaluate gravity-induced inverse

dynamics at five intermediate configurations. Torque must remain below 90% of the corresponding manufacturer limit.

- 3) **Throw-ramp torque.** The exact cubic windup-to-release velocity profile used by the runtime controller is evaluated at 40 points. In addition to inverse dynamics, the calculation includes the Jacobian-transpose feedforward term associated with the gripped object's weight. The resulting torque must remain below 75% of the corresponding limit.
- 4) **Follow-through torque and velocity.** The post-release recovery motion is evaluated over eight candidate recovery durations from 0.6 to 4.0 s, with 30 samples per duration. Both torque and joint velocity limits are checked. A candidate is accepted if at least one recovery duration satisfies the specified margins.

This cascade explicitly accounts for failure modes that are invisible to a release-state-only check. In particular, a candidate can satisfy torque constraints at release while violating them during the windup or while recovering from a high-speed release.

E. Dynamics-Aware Throw Execution

To evaluate the release state through the arm and object dynamics, the simulated manipulator is controlled using computed torque:

$$\tau = M(q)\ddot{q}_{\text{cmd}} + C(q, \dot{q}) + G(q) + J(q)^\top f_{\text{ball}}, \quad (4)$$

where M , C , and G denote the manipulator mass, Coriolis/centrifugal, and gravity terms, respectively. The final term explicitly compensates for the weight of the gripped object:

$$f_{\text{ball}} = m_{\text{ball}}[0, 0, g]^\top. \quad (5)$$

The ball is modeled as a separate rigid body attached through a removable grip constraint. The explicit feedforward term therefore accounts for the payload even though it is not part of the manipulator's kinematic chain.

Importantly, release velocity is measured from the physics engine after the grip constraint is removed rather than assigned directly. This ensures that the reported release velocity corresponds to the velocity actually produced by the simulated arm.

Three additional sources of mismatch between the learned model and the rigid-body execution model were identified during validation. First, target sampling is performed in flight space around the actual release point rather than relative to the base origin, avoiding unreachable off-axis targets. Second, the particle model uses the empirical mean release position obtained from collected trials rather than only the nominal release position. Third, the windup trajectory is explicitly rendered and checked to avoid a zero-amplitude windup configuration.

F. Height-Generalized Policy

To evaluate generalization beyond a single landing plane, both the trajectory-termination plane and particle propagation are parameterized by the target height h . The policy input

is therefore (P_x, P_y, h) . We compare three settings: a policy trained at one fixed height; separate policy re-optimization for discrete target heights while reusing the learned GP; and a single policy trained over a continuous range of target heights.

The third setting tests whether the learned model can support interpolation to target heights that are not present in the training data, without requiring a separate policy-optimization run for each height.

IV. EXPERIMENTAL SETUP

A. Simulation Environment

All arm-physics experiments use a rigid-body PyBullet simulation with the manufacturer-specified joint torque and velocity limits. The same overall pipeline is evaluated on four manipulators: KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3. This setup allows the release-state search to be evaluated under substantially different actuator envelopes without introducing **per-arm** changes to the policy formulation.

For **Gen3-specific studies**, targets are sampled in flight space, defined as an annular region around the actual release point. This avoids selecting targets that are reachable from the origin but unreachable from the actual release location.

B. Evaluation Protocol

Training-time particle cost is not used as the primary measure of throwing accuracy. Instead, every headline result is re-evaluated using the complete rigid-body rollout implemented by `PyBulletThrowingSystem.rollout` on previously unused target sets and random seeds. Unless otherwise stated, multi-seed experiments use five random seeds.

This separation between optimization and evaluation is intended to avoid reporting a low model-based training cost as evidence of physical or rigid-body landing accuracy.

C. Hardware Platform

The target hardware platform is a 7-DOF Kinova Gen3 mounted on a fixed base plate approximately 0.433 m above the floor. Joint-speed commands are sent through the Kortex SDK using `Base.SendJointSpeedsCommand`. The measured streaming rate is approximately 40 Hz.

An Intel RealSense D435i is used for perception and landing measurement, with printed ArUco/ChArUco targets providing camera-to-base extrinsic calibration. Landing positions are obtained using ray-plane intersection at known plane heights rather than relying directly on raw depth measurements.

D. Training and Implementation Details

The joint configuration and velocity are denoted by $q, \dot{q} \in \mathbb{R}^7$. The release speed is s , the release direction is $\hat{d}$, and h denotes the target height. The policy output $u \in [0, u_M]$ is obtained from the unconstrained output x using

$$u = \frac{u_M}{2} (\tanh(x) + 1). \quad (6)$$

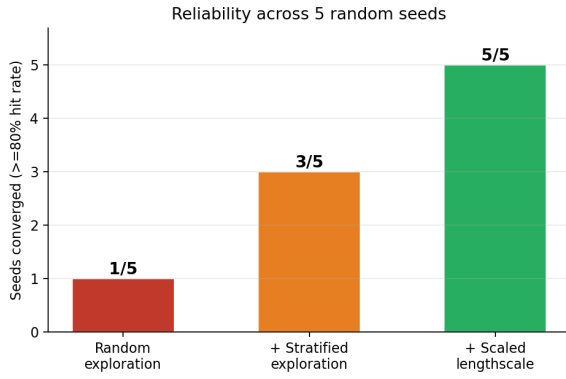


Fig. 1. Baseline optimization reliability across five random seeds before and after the exploration and lengthscale modifications. Make it this figure properly visible

The quantities $\tau_{\max}$ and $\dot{q}^{\max}$ denote the manufacturer torque and joint-velocity limits.

The RBF policy lengthscale is initialized to $0.15 \times (l_M - l_m)$, where $[l_m, l_M]$ is the target-distance range, and is subsequently trained rather than held fixed. Sparse-GP models use $M = 400$ inducing points and a mini-batch size of $N_b = 250$, following the real-setup configuration of [1] unless otherwise specified.

Simulation is executed at 50 Hz. The Gen3 hardware interface has a measured 40 Hz joint-speed streaming ceiling. Camera-to-base calibration uses exact-scale ArUco/ChArUco boards with a physical scale check before use.

V. EXPERIMENTAL RESULTS

A. Optimization Reliability

We first evaluate the reliability of the baseline optimization across multiple random seeds. The originally reported baseline result was obtained from a single seed and achieved 5/5 successful throws in five trials. When repeated across five seeds, the resulting hit rates were 60%, 20%, 80%, 10%, and 10%, demonstrating substantial sensitivity to initialization and exploration.

Inspection of the optimization procedure identified two interacting factors. Unstratified exploration can leave the GP poorly informed over parts of the release-speed range, while an RBF lengthscale that is not scaled to the target domain can saturate the policy response and reduce its variation with target position. Using stratified exploration and initializing the lengthscale relative to the target range restores convergence across all five seeds.

With these changes, evaluation over five seeds and 50 previously unseen targets per seed yields 250 throws in total. The resulting policy achieves 100% hit rate at both the < 10 cm and < 5 cm thresholds, with a mean landing error of 1.86 cm and a 95th-percentile error of 3.17 cm.

B. Cross-Manipulator Evaluation

The release-state search trains successfully on all four evaluated manipulators. Mean landing error ranges from 1.4

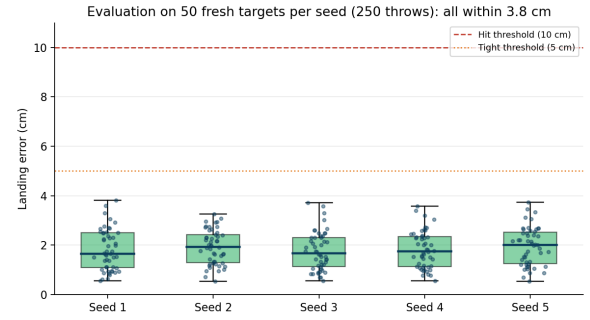


Fig. 2. Landing-error distribution after the reliability modifications, evaluated on 250 previously unseen targets across five seeds. Increase the font size of x-axis and y-axis titles, tick values, and legends so that it is visible properly

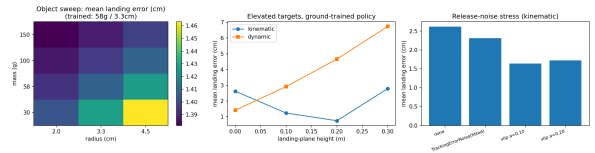


Fig. 3. Mean landing error across KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3 using the same release-state search and learning pipeline. It is not visible, so increase the font size, or you can put it separately.

to 2.7 cm without per-arm policy tuning beyond the corresponding torque and velocity limits. This result indicates that the release-state formulation is not tied to the Gen3 geometry.

We next vary payload drag from less than 1% to approximately 19% of gravity and compare the learned model against the analytical no-drag baseline. At low drag, the analytical model remains close to the observed optimum and model-based learning provides limited additional benefit. At higher drag, the analytical baseline reaches 4–6 cm error, whereas the learned model achieves 0.6–1.7 cm, corresponding to a 3–7 $\times$ improvement.

Object variation in simulation further shows approximately constant landing error of 1.4 cm over payload masses of 30–150 g and diameters of 2–4.5 cm without retraining. Under these conditions, drag remains small and the payload feedforward term directly accounts for the mass variation.

Interpretation. The observed crossover follows the expected distinction between the two models. The analytical baseline assumes drag-free ballistic motion, so its prediction error increases as aerodynamic drag contributes more strongly to the object’s flight. The learned GP instead models the realized velocity change from data. Thus, when drag is negligible, the analytical model is already sufficiently accurate; when drag becomes significant, learning the object dynamics provides a measurable advantage.

This interpretation is qualitative rather than a fitted analytical model of the crossover. Determining the exact crossover point from a finer drag sweep remains future work.

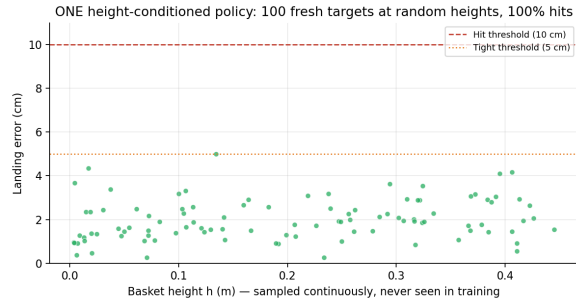


Fig. 4. Landing error versus target height for a single policy trained over a continuous height range and evaluated at previously unseen heights.

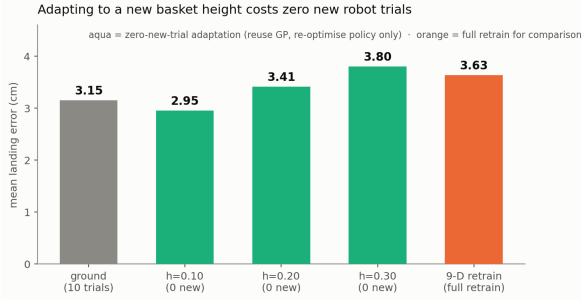


Fig. 5. Zero-new-trial adaptation to new target heights by reusing the trained GP and re-optimizing only the policy.

C. Learned Dynamics under Aerodynamic Drag

D. Target-Height Generalization

A single policy trained over a continuous target-height range is evaluated on 100 fresh targets at heights not observed during training. It achieves 100% successful throws, a mean landing error of 2.0 cm, and a worst-case error of 5.0 cm, with no **systematic degradation** over the evaluated height range.

We also reproduce the zero-new-trials height-adaptation setting by reusing the trained GP and re-optimizing only the policy for three target heights. The resulting mean errors are 2.95, 3.41, and 3.80 cm for $h = 0.10, 0.20$, and 0.30 m, respectively. No additional physical trials are required for these height changes. These values are comparable to the 3.15 cm ground-level result and outperform the 3.63 cm result obtained from a complete 9-D retraining.

E. Low-Torque Throwing and Feasible Range

For the Gen3, the fixed low-release posture does not provide a useful high-elevation throw. Increasing elevation under that posture decreases the achievable range, causing distance optimization to favor a near-horizontal motion. In contrast, searching over the release state under the complete feasibility cascade identifies a genuine overhead throwing configuration.

The selected configuration reaches a release height of approximately 1.1 m. When windup, release, and post-release recovery constraints are all enforced, the maximum simulated safe-throw distance is 0.83 m, compared with the arm's 0.87 m kinematic reach under the considered setup. This

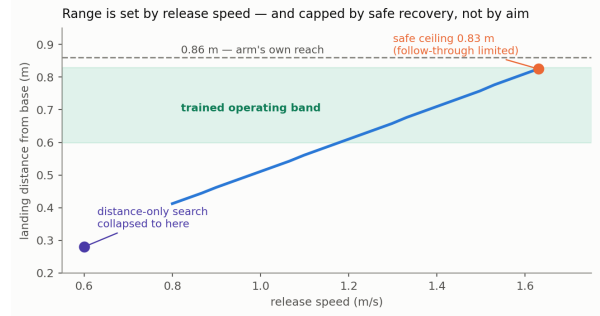


Fig. 6. Simulated safe-throw range boundary of 0.83 m compared with the 0.87 m kinematic reach of the Kinova Gen3 under the considered constraints.

TABLE I

RELEASE-INSTANT-ONLY VERSUS WHOLE-TRAJECTORY FEASIBILITY FOR THE GEN3 AND FRANKA PANDA USING THE SAME POSTURE AND DIRECTION GRID.

	Gen3	Panda
Release-instant-feasible	446,071	152,844
Also full-traj.-feasible	74,230 (16.6%)	152,844 (100%)
Rej. windup-path	242,725 (54.4%)	0
Rej. throw-ramp	66,706 (15.0%)	0
Rej. follow-through	62,410 (14.0%)	0
Best range, instant-only	1.03 m	1.65 m
Best range, full check	0.82 m	1.65 m

value is best interpreted as a measured capability boundary of the simulated low-torque platform rather than as an extended-reach result.

For this configuration, the mean landing error is 3.15 cm over 30 unseen targets, with 100% of throws within 10 cm. The release speed varies from 1.16 to 1.49 m/s over the target-distance range rather than saturating at a single maximum value.

F. Impact of Whole-Trajectory Feasibility

We quantify the effect of whole-trajectory feasibility by applying the same release-pose and direction enumeration to the Gen3 and Franka Panda. For each candidate that passes the release-instant criterion, we record whether it also satisfies the complete feasibility cascade.

For the Gen3, the complete cascade rejects 83.4% of candidates that would have been accepted by the release-instant-only criterion. The largest source of rejection is the windup-path check, which accounts for 54.4% of the release-instant-feasible candidates in the reported cascade order. The throw-ramp and follow-through stages account for 15.0% and 14.0% respectively.

The resulting best range decreases from 1.03 m under the release-instant criterion to 0.82 m after whole-trajectory validation, corresponding to a **25% reduction**. This difference demonstrates that release-state feasibility alone can overestimate the motion that the manipulator can execute and recover from.

For the Panda, none of the 152,844 release-instant-feasible candidates is rejected by the full cascade, and the best

TABLE II

REAL KINOVA GEN3 HARDWARE BRING-UP STATUS.

Stage	Status
Read-only state (joints, gripper)	live
Gripper actuation	live
Full-speed joint-velocity streaming	live
Gripper latency compensation	done
Open-loop drift budget	in budget
Loaded throw (ball in gripper)	pending
Real landing measurement	pending

candidate remains 1.65 m under both criteria. The contrast between the two manipulators indicates that whole-trajectory checking becomes most consequential when actuator torque headroom is limited.

G. Computational Efficiency

A complete Gen3 overhead-throw training run uses 10 trials, including five exploration trials, and $N_{\text{opt}} = 1500$ policy-optimization steps per trial while reusing a pre-computed release-pose table. The reported run takes 17 min 44 s on a single CPU core and uses 1.11 GB peak memory.

The one-time release-state search evaluates approximately 9×10^4 posture candidates before pruning and takes 28 min 27 s on the same single-core CPU configuration. These costs are incurred once for a given arm and workspace geometry. The resulting pose table is reused across training seeds and evaluation targets.

H. Hardware Validation

The real Kinova Gen3 is used to validate the non-destructive components of the execution pipeline. State read-back, gripper actuation, full-speed joint-velocity streaming, gripper-latency compensation, and the open-loop drift budget have been validated on the physical arm.

During bring-up, several hardware-specific effects were identified. Joint angles are reported over a $[0^\circ, 360^\circ)$ convention, requiring different handling for continuous and limited joints. Gripper-open latency was measured as 67.9 ± 6.4 ms from 1 kHz UDP feedback. At the 1.498 m/s release speed considered in the overhead configuration, an uncompensated delay would correspond to approximately 10.2 cm of travel. The executor therefore applies a wall-clock lead-time compensation.

The joint-speed streaming interface has a measured ceiling of 40 Hz. Open-loop position drift during the throw was measured as 0.0171 rad at release, corresponding to approximately 1.0 cm of landing error under the reported conversion. A remaining implementation-level contribution is the 25 ms command interval, corresponding to approximately 3.7 cm at the considered release speed.

At the current stage, the hardware validation therefore establishes the operation of the execution stack but does not establish physical throwing accuracy. In particular, the planned release speed is approximately 1.498 m/s, while the applicability of the documented 0.500 m/s Cartesian

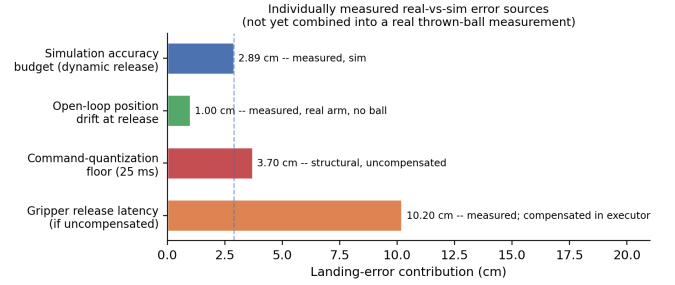


Fig. 7. Measured sim-to-real error sources identified during Gen3 bring-up, shown relative to the simulation accuracy budget. The quantities are measured independently and do not constitute a combined real throwing result.

speed ceiling to joint-speed streaming mode has not yet been established. Consequently, no real landing result is reported.

VI. DISCUSSION

The experiments support three main conclusions. First, the release configuration should not necessarily be treated as a fixed design parameter when transferring model-based throwing to manipulators with different actuator capabilities. Explicitly searching the release state allows the throwing policy to operate within the available torque and velocity envelope. Second, feasibility must be considered over the complete motion: for the Gen3, release-instant feasibility alone accepts a large fraction of candidates that later violate trajectory constraints. Third, model-based learning is most beneficial when the analytical dynamics become inaccurate, as demonstrated by the observed drag-regime crossover.

Several Limitations. should be considered when interpreting these results. First, the throwing-accuracy results are simulation-based. Although the non-destructive hardware pipeline has been validated on the Gen3, a loaded physical throw and corresponding landing measurement are not yet available. Second, the Gen3 implementation currently treats gripper delay as a measured point estimate with fixed lead-time compensation, rather than learning a distribution of release delays as in the original MC-PILOT formulation. Third, object and multi-manipulator generalization are evaluated in simulation; the real hardware study is currently limited to the Gen3. Finally, the release-state search is a discrete enumeration followed by per-candidate feasibility evaluation. It is intentionally simpler than a general continuous trajectory optimizer, but its computational cost depends on the resolution of the candidate grid.

The reported reliability analysis also highlights the importance of multi-seed evaluation in model-based policy search. A single successful optimization run can provide an incomplete picture of convergence reliability. We therefore use multiple seeds and evaluate final policies on fresh targets rather than relying on training-time particle cost.

For reproducibility, the release-state search solver, feasibility checker, multi-manipulator evaluation scripts, and trained GP/policy checkpoints for the simulation experiments are intended for public release upon acceptance. The anonymized

implementation is available to reviewers through the venue's artifact-access mechanism.

VII. CONCLUSION

We presented a constraint-aware extension of model-based robotic pick-and-throw in which the release state is selected according to the manipulator's kinematic and dynamic capabilities rather than fixed a priori. The approach combines release-state search with Gaussian-process dynamics learning and particle-based policy optimization, while evaluating torque and velocity feasibility over the complete windup, throw, and follow-through motion.

Across four simulated manipulators, the resulting framework achieves 1.4–2.7 cm mean landing error without per-arm policy tuning beyond the corresponding actuator limits. On the Kinova Gen3, whole-trajectory feasibility identifies a 0.83 m safe-throw boundary under the considered configuration and constraints. Additional experiments demonstrate continuous target-height generalization and show that model-based learning provides its largest benefit when aerodynamic drag makes a no-drag analytical model inaccurate.

The physical Gen3 experiments currently validate the non-destructive execution pipeline, including state feedback, gripper actuation, velocity streaming, and latency compensation. Physical throwing and landing measurements remain to be completed. Thus, the present results establish the simulation-side effectiveness and the hardware execution readiness of the proposed constraint-aware formulation, while leaving full real-world throwing evaluation as the next experimental step.

REFERENCES

- [1] N. Turcato, G. Giacomuzzo, M. Terrea, D. Allegro, R. Carli, and A. Dalla Libera, "Data efficient robotic object throwing with model-based reinforcement learning," *arXiv preprint arXiv:2502.05595*, 2025.
- [2] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "Tossing-bot: Learning to throw arbitrary objects with residual physics," *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.